# Short Paper: Barry: A Streaming Storage Engine for Temporal Graphs

*Anonymous submission*

## Abstract

Temporal graphs whose edges are immutable timestamped events—audit streams, agent execution traces, interaction logs—are ingested as never-ending, near-sorted streams; queried along time and causality; and retained under time-based policies. No deployed storage layer serves this combination: entity-keyed graph stores demote the timestamp to a payload attribute, columnar and LSM engines pay a join per traversal hop, versioned temporal graph stores assume few property-rich versions per entity, and statically time-sorted layouts answer queries well while being unable to ingest at all.

Barry keys storage on time itself: each timestamp maps to a virtual address by a constant-time shift, and demand paging absorbs temporal sparsity, so no index is built or maintained. Ingestion becomes a lock-free $O(1)$ append independent of arrival order; retention becomes a page release over an address prefix; compaction is explicit and off the ingest path; queries remain correct *during* concurrent ingestion through a per-slot publication protocol; and because engine state is a pure function of the edge multiset, recovery is re-ingestion. We evaluate Barry against a columnar engine, three CSR layouts, and the most recent versioned temporal graph store, with element-wise answer verification, on fourteen datasets including a billion-edge enterprise corpus, and show that keying storage on time wins the streaming and traversal axes by orders of magnitude while conceding bounded, measured costs on the structural axis.

## 1 Introduction

A growing class of graphs is built from *events*. A system call touches a file; an agent invokes a tool; a host authenticates to another; a user replies to a message. Each edge carries the moment it happened, never changes afterwards, and arrives as part of a stream that does not end. The queries these graphs exist to answer are temporal and causal: what could have influenced this state, what did this event go on to touch, what did a neighborhood look like at time $t$, how much happened in this window. And because an event's value fades with age, the data is governed by time-based retention.

Existing storage engines all approach this workload with a layout designed for something else. Dynamic graph stores [2, 3, 10, 11] key storage on source-vertex identity and treat the timestamp as payload. A time-constrained query then becomes a structural scan with a per-edge filter, and a retention pass has to find expired edges wherever vertex order scattered them. Columnar and LSM engines [9] have the opposite problem: ordered by time, they answer range filters well, but every hop of a traversal is a fresh join against the edge table. Versioned temporal graph databases [4, 7] solve a different problem—retrieving the valid *version* of an updated entity—and their structures assume few, property-rich versions per entity, which an event stream violates by construction. A statically time-sorted CSR answers temporal queries close to optimally, but it is not a storage engine: its only update path is a full rebuild, and during that rebuild it is blind.

Barry keys storage on time. The starting observation is that a 64-bit virtual address space is orders of magnitude larger than the time range of any recorded workload, so instead of maintaining an index from timestamp to location, we let the timestamp *be* the location:

$$A(T) \;=\; A_{\text{base}} + \big((T - T_{\text{start}}) \gg S\big) \times P, \tag{1}$$

one subtraction and one shift, with the operating system's demand paging absorbing temporal sparsity at no data-structure cost.

What follows from that layout is the point of the paper. An edge's home is computed, not searched, so ingestion is a wait-free slot claim. The computation ignores arrival order, so out-of-order and late events cost nothing extra. Retention is a monotone predicate on time, so expiry is the release of an address prefix. Nothing is updated in place, so crash recovery is replaying an append-only log through the normal insert path. None of these is engineered on top of the layout; each is a consequence of it.

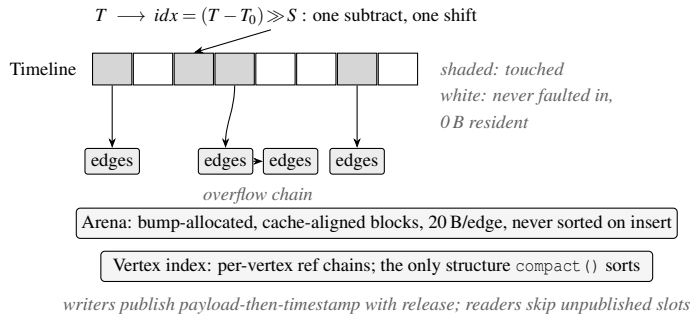



Figure 1: Barry's layout. A timestamp *computes* its timeline slot, so ingestion never searches and never sorts, and temporal sparsity is absorbed by the MMU rather than by a structure the engine maintains. Entity-keyed access lives in a separate index that compaction—never insertion—sorts.

The design also has costs, and a storage paper owes their measurement as much as their mitigation. A layout that refuses to organize by entity must pay something on entity-keyed operations. A layout that never sorts during ingestion must either scan unsorted neighborhoods or sort them later. We measure both, bound the second with an explicit compaction contract, and report the one operation where a competing design beats us outright (Section 4).

We contribute: (i) a storage design that keys temporal graphs on time, with proofs for its addressing, space, granularity, and concurrency properties (Section 2); (ii) a publication protocol under which readers never observe a torn or unpublished edge during lock-free multi-writer ingestion (Theorem 4); and (iii) an evaluation across fourteen corpora and five competing layouts that verifies element-wise answer equality before reporting any latency, separates storage cost from algorithm cost, and measures what a static structure silently loses while it rebuilds (Section 4).

## 2 Design

### 2.1 Time-keyed addressing

Equation (1) maps a timestamp to a slot in one anonymous memory mapping (Figure 1). Two properties turn that arithmetic into a storage layout.

**Theorem 1** (Addressing)**.** *For any timestamp $t$, the slot holding every edge stamped $t$ is computed in $O(1)$ worst case, with no data-dependent memory access.*

*Proof.* The map is one subtraction and one shift. Two timestamps land in the same slot exactly when $\lfloor (t - T_0)/2^S \rfloor = \lfloor (t' - T_0)/2^S \rfloor$, i.e. when they share a window, so every edge stamped $t$ is registered under that one index and no other slot can hold one. No search structure is consulted, so the bound is unconditional rather than expected-case. A B-tree or LSM index gives $O(\log E)$ with pointer chases; a hash index gives *expected* $O(1)$ with worst-case collisions. □

**Theorem 2** (Space)**.** *Let $P$ be the page size and $W_{occ}$ the number of non-empty windows. Resident timeline memory is at most $W_{occ} \cdot P$ plus page-table overhead the kernel materializes lazily; untouched windows cost zero bytes, whatever the virtual reservation.*

*Proof.* The timeline is an anonymous mapping of pointer slots. A page becomes resident only when first written, and a slot is written only when its window receives its first edge. Resident pages are therefore at most the number of pages holding at least one occupied slot, which is at most $W_{occ}$ since each occupied slot dirties one page. Untouched ranges are never faulted in and never appear in the page table. □

Sparsity is thus absorbed by the MMU, not by a structure the engine maintains. The mapping is *anonymous*, with no file behind it, so the failure modes catalogued against mmap in databases [1]—torn file pages, SIGBUS on truncation, eviction stalls—do not apply: we use mmap the way an allocator reserves address space, not as a buffer pool.

The shift $S$ sets how much time one slot covers. Too coarse and windows overfill; too fine and the timeline sprawls. Barry picks $S$ from stream statistics, and the choice is bounded.

**Theorem 3** (Granularity)**.** *For $E \geq 2\tau$ events with target occupancy $\tau$, the selected shift yields mean window occupancy $\mu \in [\tau/2, 4\tau)$.*

*Proof.* The selector computes $D = \lfloor E/\tau \rfloor$ desired windows, a desired width $w = \lfloor S_{\text{span}}/D \rfloor$, and takes the smallest $S$ with $2^S \geq w$, so $2^S \in [w, 2w)$. The realized window count $W = \lceil S_{\text{span}}/2^S \rceil$ then satisfies $D/2 < W \leq D + 1$, giving $\mu = E/W \geq E/(D+1) \geq \tau/2$ and $\mu < 2E/D < 4\tau$. □

The two factors of two are real, not slack in the analysis: one comes from rounding the window width up to a power of two, which shift addressing cannot avoid, and one from the ceiling on the window count. Across our corpora a single target yields per-window densities from 32 to 32,768 edges, and the bound holds at both ends.

### 2.2 Lock-free ingestion, correct concurrent reads

An insert computes its slot (Theorem 1), claims an index in the window's block with one `fetch_add`, and installs overflow blocks by CAS. Every failed CAS means some other insert succeeded, so ingestion is lock-free and the slot claim itself is wait-free. Nothing sorts and nothing rebalances, so arrival order does not matter—which is what separates Barry from every sorted-on-ingest design as soon as events arrive late or out of order.

Readers are protected by a publication protocol instead of locks. A writer fills an edge's payload first and stores its timestamp *last* with release semantics. Fresh arena pages are zero-filled, so a zero timestamp marks a slot that is not yet published; readers acquire-load the timestamp and skip those. Vertex-index references publish through their block pointer the same way. One extra ordering rule makes the fast read path safe as well: an append clears the bucket's *sorted* flag *before* it bumps the bucket counter.

**Theorem 4** (Concurrent reads)**.** *Under this protocol no query observes a torn, partial, or unpublished edge during concurrent multi-writer ingestion, and a reader that finds the sorted flag set may binary-search the counted prefix.*

*Proof.* For the first claim: a reader only accepts a slot whose timestamp is non-zero, and the writer's release store of the timestamp happens after its payload writes, so an acquiring reader that sees the timestamp also sees the payload. A slot is written once and never mutated, so no torn state exists after publication.

For the second: a reader loads the counter (acquire) and then the flag. Suppose it observes the flag still set. Any append that could have extended the counted prefix cleared the flag before bumping the counter, and the reader's acquire on the counter synchronizes with every bump it observes; so if any such bump were visible, the corresponding clear would be visible too and the flag would be unset. The observed prefix is therefore exactly the one compaction verified and published, and binary search over it is sound. □

## 2.3 The compaction contract

Appends leave a vertex's neighborhood in arrival order, so a time-windowed expansion has to scan it: $O(d)$. An explicit `compact()` pass sorts each neighborhood by timestamp once, and afterwards expansion binary-searches in $O(\log d + k)$ for $k$ results. Compaction is deliberately *not* on the insert path. We measured what happens when it is: an early build that compacted after every batch ingested at 11K events/s, because each batch re-walked the whole index, against 3.3M events/s once the pass was moved off the path—the same $O(\text{index})$-per-batch cost a sorted structure pays by design.

Queries are correct in both states; compaction changes their cost, not their answers. This is the paper's central trade. A statically sorted structure gets its query speed by making sorting *mandatory*, and pays for it with a rebuild window during which its answers are wrong (Section 4.3). Barry makes sorting *optional* and pays a bounded constant until it runs. The contract is: *always appendable, always correct including mid-ingest; fast expansion after a compaction you schedule when convenient.*

## 2.4 Retention and durability

Retention is a monotone predicate on time, so under Equation (1) it is an operation on an address prefix. `evict_before(`$t$`)` walks the timeline prefix, merges the block extents it finds, and returns whole pages to the OS with `madvise`; readers of other windows are untouched. There is no scan of the edge set and no tombstone compaction, because expiry and address order are the same order.

Durability uses a property most databases do not have: engine state is a pure function of the edge multiset. Edges are immutable, never updated, and deletion is retention policy rather than semantics. The log is therefore just framed, checksummed batches of raw edges; an asynchronous writer overlaps it with insertion; and recovery is replaying the log through the ordinary insert path. There is no undo phase, no redo phase, and no idempotence bookkeeping, because replaying a batch twice is indistinguishable from the multiset having contained it twice—and it cannot, since each batch is framed once. A torn tail frame fails its length or checksum test and is discarded.

# 3 Implementation

Barry is ∼6,300 lines of dependency-free C++17: the engine, nine graph algorithms including bidirectional time-respecting traversal, converters for two audit-log formats, and seven correctness suites (among them a multi-writer/multi-reader stress test of Theorem 4 and an end-to-end synthetic-provenance fixture). Interior windows of a range scan are copied without any timestamp comparison—only boundary windows can contain out-of-range edges—and boundary scans are AVX2-vectorized over the packed edges. Source, converters, proofs, and every experiment driver will be released.

# 4 Evaluation

**Setup.** 20-core Xeon, 188 GB RAM, Ubuntu 20.04. Fourteen corpora spanning five orders of magnitude in size (Table 1): LANL enterprise authentication [5] (1.05B events, 17,666 hosts, 58 days), DARPA TC whole-system provenance (CADETS 22.5M and THEIA 97.3M information-flow edges), and 12 public temporal graphs from 60K to 113M edges. Baselines: DuckDB v1.5.5 through its C API in the same process; a time-sorted CSR (`tcsr`, the strongest static competitor); an arrival-order entity CSR (`ucsr`, what an ingesting entity-keyed store holds between rebuilds); a topology-only CSR; and TVA [7] via its released artifact. Before any latency is reported, we check that the engines returned the same answer element by element. Latencies here are heavy-tailed, so we report medians with tails or results binned by answer size, never means.

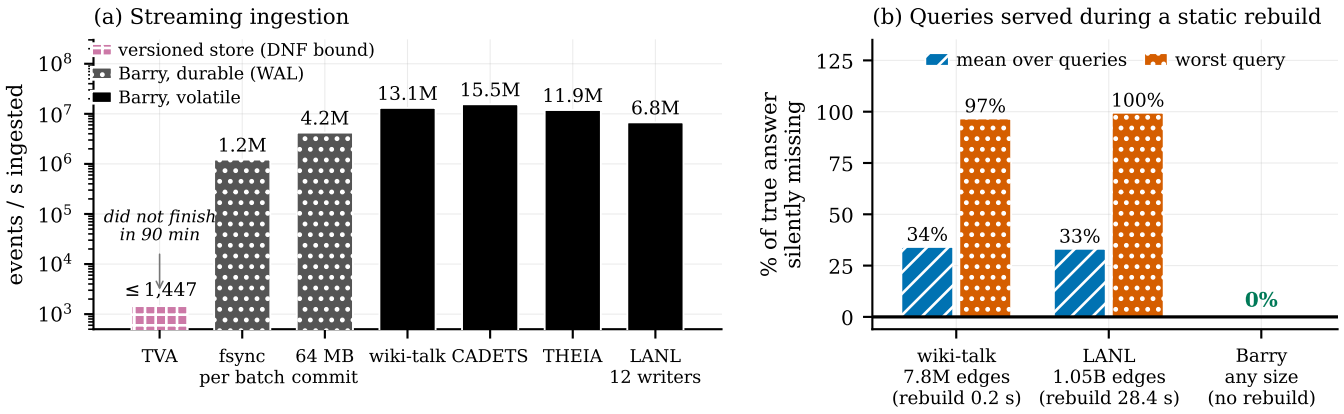



Figure 2: The dynamic axis. **(a)** Single-thread ingestion unless noted; log scale. The versioned store did not finish wiki-talk's 7.8M events within a 90-minute cutoff, so its bar is an upper bound; a statically sorted CSR has no ingest path to place on this axis at all. Barry's durable modes are bounded by the disk's 156 MB/s sequential bandwidth, not by the engine. **(b)** Queries answered from a static structure's pre-rebuild snapshot are not stale but silently incomplete; Barry never has a rebuild window, at any corpus size.

Table 1: Breadth across 13 public corpora spanning four orders of magnitude. Ingestion is what the stream sees; compaction is the one optional pass that makes neighborhood expansion binary-searchable. Traversal is the median time-respecting forward traversal over a 1% window. Resident bytes per edge are against a 20 B packed edge; the remainder is the vertex index and block headers. Eight threads.

| Corpus | Edges | $\|V\|$ | Span (d) | Ingest (Meps) | Comp. (s) | Trav. ($\mu$s) | B/ edge |
|---|---|---|---|---|---|---|---|
| CollegeMsg | 60K | 2K | 194 | 5.2 | 0.00 | 1.0 | 158 |
| email-Eu | 332K | 1K | 804 | 6.8 | 0.01 | 1.0 | 79 |
| act-mooc | 412K | 7K | 30 | 9.0 | 0.01 | 0.5 | 87 |
| mathoverflow | 507K | 25K | 2350 | 7.5 | 0.02 | 0.5 | 87 |
| askubuntu | 964K | 159K | 2614 | 6.8 | 0.04 | 0.5 | 91 |
| superuser | 1.4M | 194K | 2773 | 7.0 | 0.06 | 0.5 | 88 |
| wiki-talk | 7.8M | 1.1M | 2320 | 6.2 | 0.34 | 0.5 | 68 |
| cit-Patents | 16.5M | 6.0M | 1826 | 7.4 | 0.76 | 0.5 | 77 |
| TC-CADETS | 22.5M | 2.9M | 11 | 5.8 | 0.86 | 0.5 | 65 |
| stackoverflow | 63.5M | 2.6M | 2774 | 5.8 | 3.11 | 0.5 | 66 |
| LiveJournal | 69.0M | 4.8M | 1157 | 5.3 | 3.50 | 0.7 | 71 |
| TC-THEIA | 97.3M | 6.9M | 10 | 4.9 | 3.45 | 0.5 | 63 |
| bitcoin | 113.1M | 48.1M | 2585 | 1.9 | 5.96 | 0.5 | 107 |
| *plus* LANL, 1.05B edges (§4.1, §4.3) | | | | | | | |

## 4.1 Ingestion

Across the 13 corpora of Table 1, Barry ingests at 1.9–9.0M events/s on eight threads, and the one optional compaction pass costs at most 6.0 s on the largest of them (113M edges). Single-threaded, matched against the competing engine's own thread count, the rates are 11.9–15.5M events/s. On the billion-edge LANL corpus Barry sustains 6.8M events/s under 12 concurrent writers *while* four readers issue validated queries: 48,636 queries, 780M delivered edges checked, zero protocol violations, and a final edge count that matches exactly. With the write-ahead log on, durable ingestion reaches 4.2M events/s under 64 MB group commit against 7.4M volatile—the limit there is the disk's measured 156 MB/s sequential bandwidth, not the engine—and 1.2M with an fsync per batch.

Driving TVA's artifact with the same single-threaded event stream did not complete wiki-talk's 7.8M events within a 90-minute cutoff, at 41.6 GB RSS (Figure 2a). The cause is structural rather than incidental: in an event stream every repeated $(src, dst)$ contact extends one edge's version chain, which is exactly the structure its hopscotch layout must keep within bounded offsets, so its design assumption—few, property-rich versions per entity—inverts. A statically sorted CSR is absent from this comparison for a simpler reason: it has no ingest path to measure.

## 4.2 Time-respecting traversal

On multi-hop time-respecting traversal run to fixpoint, Barry leads DuckDB by 150–700× across all answer sizes; at $|R| > 1{,}000$ the median is 12 ms against 2.43 s (Figure 3a). The reason is structural: each hop costs a relational engine a join against the edge table. The time-sorted CSR stays 2.7× faster than Barry at high degree, which is the measured price of refusing to sort during ingestion; Section 4.3 prices what the CSR pays for that privilege.

Against the *ingesting* entity-keyed layout the ordering flips with graph density (Figure 3b). We normalise to cost per delivered edge, because the corpora span 60K to 1.05B edges and absolute times are not comparable across them. An arrival-order adjacency has to scan a whole neighborhood to find the edges inside the query window, so its per-

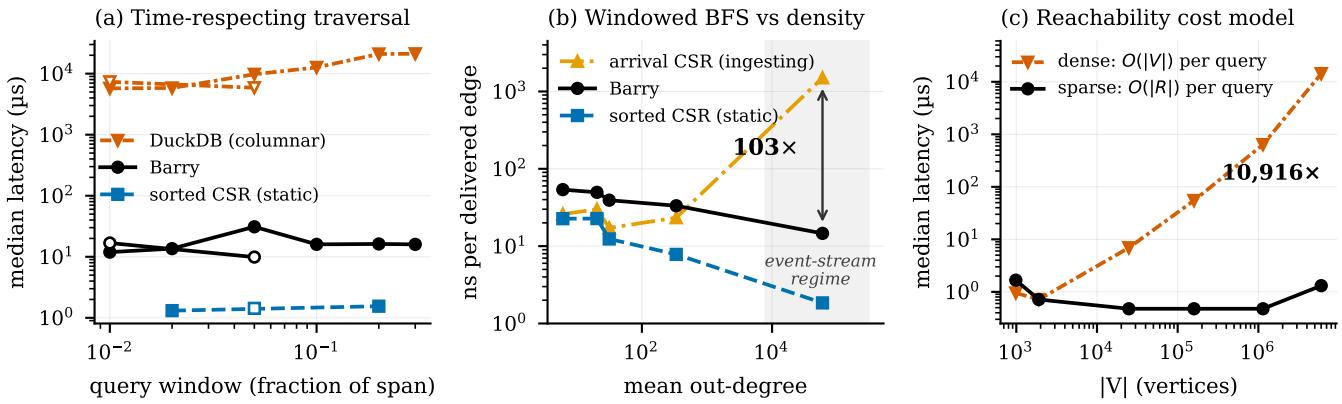



Figure 3: The time axis, three ways. **(a)** Time-respecting traversal to fixpoint across query windows on two corpora (filled markers: sx-mathoverflow; open: wiki-talk); answers verified element-wise identical across engines before timing. **(b)** Windowed BFS, normalised to cost *per delivered edge*—the only size-independent comparison across corpora spanning 60K to 1.05B edges. An arrival-order adjacency must scan an entire neighborhood to find the in-window edges, so its per-edge cost explodes with density (26 → 1500 ns); Barry's compacted buckets amortise a binary search, so its cost *improves* (54 → 15 ns). The crossover lands inside the regime event streams occupy. **(c)** Sparse temporal reachability makes query cost track the answer rather than the graph.

edge cost rises from 26 ns at degree 6 to 1,500 ns at degree 59,517. Barry's compacted buckets amortise a binary search over the same neighborhood, so its per-edge cost *falls* from 54 ns to 15 ns. The crossover sits near degree 100, inside the range event-stream workloads occupy.

Two more results bound the picture. On time-*range* filtering, DuckDB's home ground, Barry's vectorized scan wins the median 10.6× (77 µs against 819 µs over five seeds) with p95 at parity (981 against 1,105 µs, overlapping ranges). And on structure-only algorithms we decompose each run into the share spent delivering adjacency, under executions verified identical by checksum. Over 29 measurements on 11 corpora, Barry delivers at 0.9–5.8× the topology-only CSR optimum, median 2.1×, and the spread separates into two regimes (Figure 4). Frontier-driven traversal touches only the vertices it reaches and stays at 1.4–2.2× regardless of corpus size, including 1.4× on the billion-edge corpus. Whole-graph iteration such as PageRank or label propagation walks every vertex's bucket chain on every round, where a CSR walks one contiguous array, and that gap widens with size: 5.8× at 69M edges. This is the concession, it is the price of not organising by entity, and it buys an ingest path the optimum does not have.

Barry's weakest operation is the full-graph as-of snapshot, which it answers by enumerating and reducing: 218 s on CADETS, whose hottest vertices carry millions of events. That is precisely the operation TVA's version-skipping is built for, and we report it as its win.

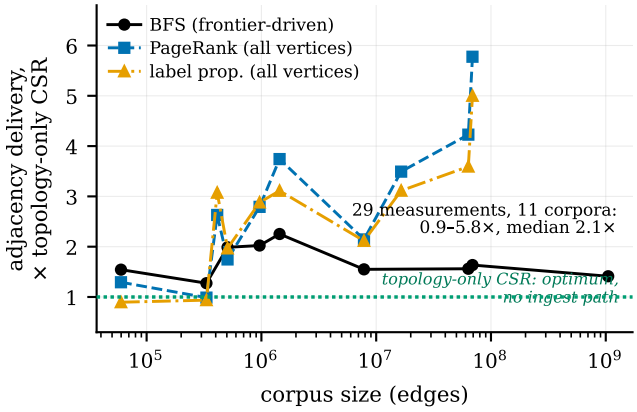



Figure 4: The concession, priced. Adjacency delivery for structure-only algorithms against the topology-only CSR optimum, under identical checksum-verified executions, across 11 corpora. The penalty separates by access pattern: frontier-driven traversal is flat in corpus size, whole-graph iteration is not, because it walks every bucket chain on every round.

## 4.3 Rebuild versus stream

The time-sorted CSR's only update path is a full rebuild: 28.4 s on LANL. A query about the live window served from the pre-rebuild snapshot is not stale but silently *wrong*. On LANL, 68 of 100 such queries returned incomplete answers, losing 33.2% of the true result on average and 99.5% in the worst case; an independent wiki-talk measurement gives 34.1% and 96.7% (Figure 2b). Barry ingests the same arrivals incrementally at 3.3M events/s and has no blind win-

dow at all.

Barry's own optional compaction costs more than the CSR's rebuild on LANL (76 s against 28 s), because it resolves a timestamp per reference by chasing a pointer into the arena; resolving in arena order instead would close most of that, and is future work. But the comparison is between two different kinds of cost. The CSR's rebuild is on the correctness path—without it the answers are wrong. Barry's compaction is on the performance path—without it the answers are right and expansion is slower. That difference is the whole point of the contract.

### 4.4 Retention, scale, and a real workload

`evict_before` releases 90% of a 16.5M-edge history in 18 ms and returns 331 MB of resident memory, 5–29× faster than a columnar `DELETE`. Both reclaim memory fully, so the win here is speed alone—we had expected the columnar engine to mark rows without reclaiming, and it does not. Across the corpora of Table 1 resident memory is 63–158 B per edge against a 20 B packed edge; the rest is the vertex index and block headers. LANL loads in 255 s at 96 GB RSS with the reverse index enabled.

As an end-to-end check on real ground truth: tracing all 749 labelled red-team alerts of the LANL corpus to causal fixpoint takes 46.5 s in total (median 24.6 $\mu$s, p95 353 ms). On DARPA CADETS, a forward trace from the documented exploit source reconstructs the published attack chain—implant drop, privilege escalation, C2, injection target—in causal order, inside an impact set that is 0.0093% of the graph, at 133 $\mu$s.

### 4.5 Negative results, kept

For the record: caching timestamps in vertex-index refs (+50% index memory for 15% traversal) was built, measured, and reverted; binary-search expansion buys 2.5× at degree 337 and nothing below degree 30; and the retention comparison's original hypothesis—that the columnar engine would fail to reclaim memory—was false.

## 5 Related Work

**Versioned temporal graph stores.** TVA [7] and AeonG [4] organize versions of *updated entities* for snapshot retrieval; delta encoding and version-chain locality, their central difficulties, do not arise in an append-only event stream, and time-respecting reachability is not expressible over their snapshot query surface. The two lines are complementary, and measurably so in both directions (Sections 4.1 and 4.2): neither subsumes the other; they store different things. **Structural dynamic graph stores** [2, 3, 10, 11] optimize the entity axis and treat time as payload; the storage-read decomposition bounds what Barry concedes on their axis, and none answers time-respecting queries natively. **Columnar/LSM engines** [9] own predictable range-filter tails and lose traversal by orders of magnitude. **mmap in databases.** The case against mmap [1] targets file-backed buffer management; Barry's anonymous reservation sidesteps each cited failure mode. **Provenance systems** [6, 8, 12] motivate the workload's rates and retention stakes; serving them is an application of this engine, developed separately.

## 6 Conclusion

Keying graph storage on time collapses the machinery an event stream otherwise requires: the index becomes a shift, ingestion an order-indifferent lock-free append, retention a page release, recovery a replay, and temporal traversal an address computation—correct even mid-ingest. The concessions are bounded constants, measured at the storage layer where they belong; the wins are the axes an event stream cannot live without.

## References


[1] Andrew Crotty, Viktor Leis, and Andrew Pavlo. Are you sure you want to use MMAP in your database management system? In *Proc. CIDR*, 2022.

[2] Laxman Dhulipala, Guy E. Blelloch, and Julian Shun. Low-latency graph streaming using compressed purely-functional trees. In *PLDI*, 2019.

[3] Per Fuchs, Domagoj Margan, and Jana Giceva. Sortledton: A universal graph data structure. *ACM SIGMOD Record*, 2023.

[4] Jiamin Hou, Zhanhao Zhao, Zhouyu Wang, Wei Lu, Guodong Jin, Dong Wen, and Xiaoyong Du. Aeong: An efficient built-in temporal support in graph databases. *PVLDB*, 17(6), 2024.

[5] Alexander D. Kent. Comprehensive, Multi-Source Cyber-Security Events. Los Alamos National Laboratory, 2015.

[6] Samuel T. King and Peter M. Chen. Backtracking intrusions. In *Proc. SOSP*, 2003.

[7] Wenhao Li, Zhanhao Zhao, Jinhao Dong, Jiamin Hou, Wei Lu, Yunhai Wang, and Xiaoyong Du. TVA: A version-aware temporal graph storage system for real-time analytics. In *PVLDB*, volume 19, 2026.

[8] Thomas Pasquier, Xueyuan Han, Mark Goldstein, Thomas Moyer, David Eyers, Margo Seltzer, and Jean Bacon. Practical whole-system provenance capture. In *Proc. SoCC*, 2017.

[9] Mark Raasveldt and Hannes Mühleisen. DuckDB: An embeddable analytical database. In *Proc. SIGMOD*, 2019.

[10] Yue Shen, Jie Zhang, Huawei Cao, Yuan Zhang, and Xuejun An. Dcsr: A fast data structure with leaf-oriented locks for streaming graph processing. In *EDBT*, 2026.

[11] Brian Wheatman and Helen Xu. A parallel packed memory array to store dynamic graphs. In *ALENEX*, 2021.

[12] Bo Zhang et al. TAPAS: An efficient online APT detection with task-guided process provenance graph segmentation and analysis. In *Proc. USENIX Security*, 2025.